CPO-STCM software for partially observed consistency in LED animation anomaly detection

Xiaoxue Su[1,2], **Wanli Wang**[1,2*]

Xiaoxue Su: xiaoxuesu@jcut.edu.cn; Wanli Wang: wangwanli@jcut.edu.cn *Corresponding author.

1 School of Artificial Intelligence, Jingchu University of Technology, Jingmen 448000, China

2 Intelligent Information Technology Research Center, Jingchu University of Technology, Jingmen 448000, China

Abstract

CPO-STCM is open-source Python software for normal-only anomaly detection in LED-screen animations. It combines a CNN-LSTM temporal autoencoder with partially observed semantic-slot consistency and block-aware conformal calibration. The package includes a command-line interface, PEP 517 packaging, automated tests, pinned public-dataset access, and a reproducible path from CPO-LED Bench images to training arrays. CPO-LED Bench v2.0 provides 70,140 images, 13,802 temporal windows, and production-derived anomalies. We describe the software architecture, installation, reproducibility workflow, and impact, and state current limitations, including capture-profile sensitivity and the optional AGPL-licensed static detector.

Keywords: Anomaly detection; Conformal prediction; Industrial inspection; LED animation; Normal-only learning; Reproducibility

1. Code metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v1.0.0-rc.1
C2	Permanent link to code/repository used for this code version	https://github.com/wwlswj/CPO-STCM
C3	Permanent link to Reproducible Capsule	https://doi.org/10.5281/zenodo.24031262
C4	Legal Code License	AGPL-3.0-only
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python, PyTorch, NumPy, SciPy, scikit-learn, OpenCV, optional Ultralytics
C7	Compilation requirements, operating environments and dependencies	Python 3.10-3.12; Windows and Linux; CPU baseline with optional NVIDIA CUDA; dependencies pinned by uv.lock; CPO-LED Bench v2.0.0 archive 395563944 bytes, SHA-256 a462f4c144b4675aaeb7ccdc819066653fd9500615843996f941226c05a3b19d; optional Docker image
C8	If available, link to developer documentation/manual	https://github.com/wwlswj/CPO-STCM/tree/main/docs
C9	Support email for questions	wangwanli@jcut.edu.cn

Motivation and significance

LED screens communicate charging, shutdown, and fault states through short animations. Their meaning depends on both spatial layout and temporal order: a valid symbol may appear in the wrong sequence, a device may leave the camera view, or an object detector may miss a low-contrast display element. Normal-only inspection is therefore difficult when the implementation and the published evaluation protocol are separated.

CPO-STCM packages the operational pieces required to address this problem. The temporal branch learns normal difference-image transitions and scores reconstruction and ordering evidence. The static branch maps detection outputs to semantic slots and evaluates weighted isotonic consistency under optional observation. Block-aware conformal calibration converts anomaly evidence into false-alarm-controlled decisions. The software retains these method components while exposing data preparation, model execution, stress evaluation, and frozen reporting as a reusable workflow [1,2].

Software description

The Python package separates lightweight calibration utilities from optional neural-network and detector dependencies. Importing cpo_stcm does not load PyTorch. Users can run a deterministic conformal demo, inspect resolved paths, verify the public dataset, and query the command-line interface in a base CPU installation. The ml extra adds PyTorch, OpenCV, pandas, and matplotlib. The static extra adds Ultralytics for optional detector recomputation [5].

The command cpo-stcm data fetch downloads CPO-LED Bench through an atomic partial-file workflow, checks the fixed archive size and SHA-256, and rejects unsafe ZIP members. The command cpo-stcm data build-packed reconstructs the training-time memory-mapped array from the published PNG image index. The array uses the stable packed_index from the source manifest and validates frame count and image dimensions for every window.

Training, inference, joint evaluation, natural-fault analysis, static-deletion stress, and external UCSD Ped2 experiments are exposed through cpo-stcm workflow. The workflows retain explicit command-line arguments and write machine-readable JSON reports. The release repository also contains a PEP 517 build configuration, type marker, unit tests, CI definition, container definition, citation metadata, third-party notices, and data and model limitations.

Reproducibility and validation

CPO-LED Bench v2.0 is a public image-inclusive benchmark containing 70,140 image files, 5,010 spatial base windows, 13,802 expanded temporal windows, and 222 production-derived natural anomalies [3]. The v2.0.0 archive remains immutable. The v2.0.1 metadata revision corrects the dataset author order and is archived under a persistent DOI without changing image, annotation, split, model, or archive content.

The software was tested in independent Python 3.12.10 environments without system site packages. The base suite passed 24 tests with three optional PyTorch tests skipped; the CPU ML suite passed all 24 tests with PyTorch 2.15.0+cpu. Ruff, compile checks, dependency checks, source-distribution and wheel builds passed. A locked environment generated from uv.lock reproduced the full 24-test result. Packing all published dynamic images produced an array of shape 7322 x 10 x 100 x 300, containing 5,010 valid windows in 2,196,600,128 bytes with SHA-256 58e84979e4c10e925c0577e592293730f486108a67c2693882dfa29f3f66e02b.

The v1.0.0-rc.1 software release is archived under the version DOI 10.5281/zenodo.24031262 and the concept DOI 10.5281/zenodo.24031261. The record binds the Git tag, wheel, source archive, release notes, and SHA-256 manifest. The software paper should be submitted only after the Pattern Recognition paper cited in the enabled-publications list has reached its final publication status.

Impact overview

CPO-STCM enables research questions that require joint analysis of temporal order and partial visibility. Investigators can replace a detector, calibrate normal-only alarm budgets, isolate source sequences, and compare reconstruction, order, semantic-layout, and fused evidence in one reproducible pipeline.

The software also improves existing practice. A user does not need to reconstruct repository-specific paths, infer manifest semantics from filenames, or manually coordinate plotting and evaluation scripts. The public data checksum, image-to-array conversion, explicit workflow arguments, and frozen JSON reports reduce opportunities for silent protocol drift. Researchers can reuse the package as a command-line tool, import calibration and evaluation functions as a library, or adapt the workflow modules to another animation domain.

The demonstrated technical impact is the measured capability of the accompanying method. At a nominal 5% alarm budget, the charge branch obtained AUROC 0.927 and F1 0.870 on mixed data; observability-aware calibration increased F1 to 0.873 and reduced the partial-observation false-positive rate from 0.117 to 0.078. Production-derived anomalies reached AUROC 0.993 and F1 0.972 [2]. These values demonstrate software-mediated analysis; they are not evidence of external adoption.

External uptake, community reuse, and commercial deployment have not yet been measured. The open repository, dataset DOI, software DOI, executable examples, and documented workflow provide a citable basis for such evaluation.

Limitations and future improvements

The default geometry assumes a one-row, four-device LED array and ten-frame animation windows. The static model expects nine calibrated semantic slots. Capture-profile stress produced a false-positive rate of 0.346, so profile-aware calibration and additional profile-specific normal data remain necessary.

The public image archive does not contain every original full-size raw frame needed to recompute static detections. The software therefore supports frozen public static evidence, while complete detector recomputation requires an additional raw-frame dataset. The optional Ultralytics path is distributed under AGPL-3.0 terms [5].

Future work should add generic display-layout adapters, an independently licensed detector backend, real-time camera integration, multi-site validation, and a signed OCI image. Until those extensions are validated, the software should not be described as a safety-critical or zero-failure inspection system.

Scholarly publications enabled by the software

The software directly supports: W. Wang, H. Chen, and X. Wei, Partially Observed Weighted Isotonic Conformal Consistency for Normal-Only Animation Anomaly Detection, Pattern Recognition, manuscript under review [2]. Its journal citation and DOI should be inserted after acceptance. No other enabled publication is claimed at this stage.

References

[1] V. Vovk, A. Gammerman, and G. Shafer, Algorithmic Learning in a Random World. Springer, 2005. doi:10.1007/b106715.
[2] W. Wang, H. Chen, and X. Wei, Partially Observed Weighted Isotonic Conformal Consistency for Normal-Only Animation Anomaly Detection, Pattern Recognition, manuscript under review.
[3] W. Wang, H. Chen, and X. Wei, CPO-LED Bench v2.0.1, dataset, 2026. https://doi.org/10.5281/zenodo.24031281
[4] X. Su and W. Wang, CPO-STCM v1.0.0-rc.1, software, 2026. https://doi.org/10.5281/zenodo.24031262
[5] Ultralytics, YOLO licenses and AGPL-3.0 terms. https://www.ultralytics.com/license (accessed 7 November 2026).